GOLDENHOUR / TECHNICAL EXPLAINER

The stack. The techniques. The reasons.

How the current project is built

A source-based explanation of each technology, how it is used, why it fits, and where the current implementation ends.

01

INTERFACE

React, TypeScript and Tailwind

02

API + DATA

Fastify, contracts and current storage

03

DECISIONS

AI assistance and deterministic logic

04

COORDINATION

Maps, simulation and live events

Read this as a current-state guide
Implemented source, infrastructure-only dependencies and planned features are identified separately. Technology choices are explained through their actual role in GoldenHour.

Prepared 24 October 2026
Based on repository inspection and primary provider documentation.
No application code, tests or deployment were executed for this review.

Guide contents

Start with the architecture, then explore each technology and engineering technique. The final sections provide an honest feature-status checklist and presentation answers.

1. What GoldenHour does 3

2. Architecture and the path through the system 4

3. Frontend technologies: what, how and why 5

4. Backend, contracts and configuration 7

5. Authentication and authorization techniques 8

6. AI assistance and deterministic decision techniques 9

7. Maps, routing and simulation 10

8. Data, consistency and realtime coordination 11

9. Testing, build and deployment tools 13

10. Current versus planned: presentation checklist 14

11. Explain the project in an interview or demonstration 15

12. Source map and supporting references 16

Reading guide

Quick project explanation: sections 1, 2 and 11.
Technology-by-technology detail: sections 3-9.
What is actually implemented: section 10.

1. What GoldenHour does

GoldenHour is an emergency hospital coordination prototype. An ambulance crew enters observations, reviews proposed requirements, compares hospital destinations and coordinates acceptance and resource use. Hospitals manage receiving status, resource pools and requests. Dispatchers inspect cases, facilities, audits and simulation behavior.

A short project explanation: GoldenHour combines a typed web application, an API service, AI-assisted requirement interpretation, deterministic hospital filtering and route-based ranking. Human users confirm the important decisions. Live updates connect the ambulance, hospital and dispatcher screens.

This guide describes the source inspected on 24 October 2026. Package versions are declared ranges from package manifests, not a claim about the exact installed dependency resolution. The reasons given are engineering explanations of why each choice fits this codebase; they are not historical statements from the original authors.

What was verified

Source inspection covered package manifests, frontend pages/components, backend modules, shared contracts, authentication, routing, simulation, database support, tests and deployment configuration. No application tests, external model calls, migrations or deployment were executed for this document.

Status vocabulary

- **Implemented in source:** A current code path exists. This does not mean it has been exercised against live services in this review.
- **Infrastructure present:** Dependency or configuration exists, but the full intended behavior is not established.
- **Planned:** Described in earlier Markdown proposals and not found as an implemented runtime module in this review.

2. Architecture and the path through the system

```
React browser application
  Ambulance | Hospital | Dispatch
       | HTTP/JSON requests      | Socket.IO updates
       v                        ^
Fastify API on Node.js
  Auth -> validation -> domain services -> current in-memory store
  Agent 1 -> Gemini API or deterministic rules
  Agent 2 -> hospital capability/resource filtering
  Agent 3 -> graph routing and destination ranking
  Routing adapter -> Google Compute Routes when configured
  Simulation -> server-side movement and telemetry

PostgreSQL schema, migration and connection support are present;
the inspected business store still keeps its main entities in Maps.
```

Why a modular monorepo fits

The repository contains `apps/web`, `apps/api`, `packages/contracts` and `packages/scenario-fixtures`. They share one workspace but have distinct responsibilities. Shared contracts reduce disagreement between client and server, while shared fixtures make the same hospital and road scenarios available to the backend and tests.

The backend is a modular application, not a set of independently deployed microservices. The three agents are responsibility boundaries, not three separate infrastructure services. This keeps initial development, debugging and demonstrations manageable.

A typical case journey

1. The browser submits observations through the typed API client.
2. Agent 1 proposes full-care and stabilization requirements, using Gemini when available and a rules fallback otherwise.
3. A clinician reviews and confirms requirements.
4. Agent 2 evaluates participating hospitals; Agent 3 ranks candidates.
5. The current source supports full-care, stabilization-only and no-eligible-destination modes.
6. The crew selects a destination. The selected hospital receives a MAIN request; alternatives can receive standby requests.
7. Hospital acceptance checks required resources and records reservations. Socket events update interested screens.
8. Diversion, arrival, handover and explicit equipment release follow separate lifecycle operations.

3. Frontend technologies: what, how and why

React and React DOM - declared 18.3.1

What: React organizes an interactive interface into components; React DOM renders those components in the browser.

How used: `App.tsx` hosts the application; Ambulance, Hospital and Admin pages compose forms, recommendations, maps and lifecycle controls. Hooks hold selected cases, pending requests, errors and fetched data. Reusable components include the header, API health indicator, recommendation panel and deterioration dialog.

Why it fits: The same information changes in several places as requests and telemetry arrive. Declarative rendering makes these updates easier to coordinate than manually editing DOM elements throughout the application.

Tradeoff: Component state and asynchronous effects can drift if dependencies or cleanup are wrong. React does not automatically solve authentication races, stale requests or socket authorization.

TypeScript - declared 5.7.3

What: JavaScript with compile-time type checking.

How used: Both apps and shared packages use TypeScript. DTOs such as hospital, case, recommendation and allocation shapes guide function signatures and component props.

Why it fits: Changes to a hospital or dispatch contract affect several modules. Static checks catch many mismatched fields before execution and make refactoring easier.

Tradeoff: Types disappear at runtime. A cast such as `data as T` does not verify a network response; `any` weakens guarantees. Zod provides a separate runtime validation layer where invoked.

Vite and the React plugin - declared Vite 6.1.1

What: Frontend development server and production asset build pipeline.

How used: `vite.config.ts` serves the web app on port 3000 and proxies API, health and Socket.IO requests to the backend. The web build runs TypeScript checking followed by Vite's build.

Why it fits: The project is a browser application with a separate API, so a lightweight development/build tool avoids introducing an unnecessary server-rendering framework.

Tradeoff: Public Vite variables become browser-visible build configuration. They cannot hold backend secrets. Changes to those values usually require rebuilding the frontend artifact.

React Router - declared 7.1.5

What: Client-side URL routing.

How used: Browser routes select the ambulance, hospital and admin pages; `/ambulance` redirects to the main ambulance route.

Why it fits: Each role workspace can have a distinct URL while remaining in one application shell. Navigation does not require downloading a new document for every screen.

Tradeoff: Hiding a route or button is not access control. The API must enforce authorization independently, and the web server needs a single-page-app fallback for direct navigation.

Tailwind CSS, PostCSS and Autoprefixer

What: Utility-based styling plus CSS build processing. Declared versions include Tailwind 3.4.17, PostCSS 8.5.3 and Autoprefixer 10.4.20.

How used: Components use utility classes for spacing, responsive grids, borders, typography and state styling. `index.css` and Tailwind configuration establish shared presentation behavior. Many current components also contain explicit teal/white color values.

Why it fits: Utility classes let the team iterate quickly on data-heavy layouts without maintaining a separate stylesheet for every component.

Tradeoff: Repeated literal colors and long class strings can create inconsistency. Shared tokens and components still matter. Autoprefixer processes CSS compatibility; it does not guarantee every layout works on every browser.

Lucide React - declared 0.475.0

What: React SVG icon library.

How used: Navigation, hospital, ambulance, warning, route and action controls use a consistent icon family.

Why it fits: Reusable vector icons remain sharp at different sizes and are easier to keep visually consistent than mixed emoji or unrelated image assets.

Tradeoff: Icons need accessible text or labels. An icon alone does not explain a consequential action.

Context, Fetch and browser APIs

`AuthContext` supplies current user and login/logout/role-switch behavior. The API client wraps browser `fetch`, sends cookies with `credentials: include`, parses JSON and raises a structured `ApiError` for failed responses. This centralizes request mechanics instead of duplicating them in every page.

Browser Geolocation supplies device coordinates when the user opts in. It is a position source, not a hospital-routing service. A device's GPS position must be associated with the correct ambulance; background reliability and accuracy are separate concerns.

The `clsx` and `tailwind-merge` packages are declared utilities for conditional classes and resolving Tailwind class conflicts. The inspected source search did not establish their active use, so present them as available dependencies rather than core implemented techniques.

4. Backend, contracts and configuration

Node.js and ES modules

What: Node runs JavaScript/TypeScript-derived server code; the packages declare ES module syntax.

How used: The API listens for HTTP requests, manages Socket.IO, calls Gemini and routing services, hashes passwords and runs simulation timers. Dockerfiles use a Node 20 base; that is deployment configuration, not a statement that Node 20 is the newest runtime.

Why it fits: One language across browser, server and contracts reduces translation overhead. Asynchronous I/O fits API requests and external-service calls.

Tradeoff: Heavy CPU work can block the event loop. In-process Maps, timers and sessions are local to one process and do not automatically scale across replicas.

Fastify - declared 5.2.1

What: HTTP API framework with route registration, hooks and plugins.

How used: `server.ts` registers health, authentication, hospital, case, dispatch, admin and simulation modules. Handlers apply authentication/ownership checks, parse requests and return JSON/status codes.

Why it fits: Domain routes remain grouped, and common behavior can be attached through hooks rather than copied into every handler. The server factory also supports isolated API testing.

Tradeoff: The framework does not make authorization uniform automatically. Each route and socket path must be checked for the correct role and entity scope.

Zod and shared contracts - declared 3.24.2

What: Runtime schemas that can also produce TypeScript types through inference.

How used: `packages/contracts` defines requests, DTOs, enums and validation. API handlers use schemas such as dispatch, diversion, hospital-update and simulation requests.

Why it fits: A single shape can describe what the browser sends and what the server accepts. Runtime checks catch invalid numbers, missing fields and unsupported enum values that TypeScript alone cannot catch from external input.

Tradeoff: Schema validation checks structure, not the whole business meaning. A valid hospital ID still needs existence, authorization, resource and lifecycle checks. Shared DTOs also do not prove that every response is parsed at runtime.

Supporting Fastify plugins

- `@fastify/cookie`: reads/writes session cookies for the authentication flow.
- `@fastify/cors`: configures cross-origin requests. Current registration permits origins broadly with credentials, so do not describe it as a restrictive production allowlist.
- `@fastify/sensible`: provides common server conveniences and HTTP-oriented utilities.
- `fastify-plugin`: supports plugin encapsulation/dependency behavior in the authentication integration.

These reduce infrastructure boilerplate. They are supporting tools, not the domain logic that selects or reserves a hospital.

dotenv, tsx and configuration validation

`dotenv` loads environment values. `config.ts` attempts to validate settings with Zod and supplies defaults. `tsx watch` runs TypeScript during API development; production build scripts use `tsc` to compile it.

This separates credentials and deployment settings from business logic. The current development fallbacks and environment examples should not be described as production secret management. No real secret values are reproduced in this guide.

5. Authentication and authorization techniques

Salted scrypt password hashing

`modules/auth/password.ts` uses Node's `crypto.randomBytes` to create a salt, `crypto.scrypt` to derive a password hash, and `timingSafeEqual` to compare derived values.

Why: A slow password-derivation function is more appropriate than storing plaintext or using a fast general-purpose hash. Per-password salts prevent identical passwords from automatically producing identical stored values. Timing-safe comparison avoids a straightforward comparison-time leak. See Node crypto documentation.

Limit: This is a credential-storage technique, not proof of a complete security assessment. Rate limiting, deployment configuration and access reviews remain separate work.

Opaque cookie sessions

`session.ts` generates a random session identifier, stores session data in a JavaScript Map and applies a 24-hour expiry. The browser sends a cookie; the server resolves it to a user.

Why: Session identifiers keep authentication state under server control and allow server-side logout/invalidation within the running process.

Limit: Sessions disappear on restart and are not shared across API replicas. Although a JWT-related environment variable appears in deployment configuration, the inspected authentication mechanism is opaque sessions, not a JWT-based login implementation.

Role and ownership checks

Authentication answers “who is this user?” Role checks answer “may this role perform this operation?” Ownership checks answer “may this hospital user change this particular hospital's resource?” These are different layers.

The code contains authentication and ownership helpers, and hospital mutation routes use them. Do not infer that all access paths are equally restricted: realtime demo behavior, broad CORS and individually configured routes need their own verification.

6. AI assistance and deterministic decision techniques

Agent 1: Gemini plus a rules fallback

Technology: `@google/genai`, declared 2.31.0, behind a `RequirementsInterpreter` interface.

How used: `runAgent1Async` delegates to an interpreter. The Gemini path sends observations and a prompt requesting JSON requirement arrays, reasons and missing inputs. The inspected source names `gemini-3.5-flash-lite`; that is a configured model string, not proof of account availability or a successful live call.

Why it fits: A language model can interpret unstructured notes and produce a readable proposal. The interface separates the caller from the provider, allowing a deterministic fallback when the key is absent or generation fails.

Techniques: Prompt constraints, low generation temperature, JSON output mode, bounded waiting through a timeout, and graceful fallback.

Limits: JSON MIME mode is not equivalent to validated clinical correctness. Current parsing checks selected arrays rather than establishing complete semantic evidence. The fixed confidence numbers 0.95 and 1.0 are not measured clinical accuracy. A Promise timeout does not itself prove the external request was cancelled.

Current RAG status: Agent 1 does not currently retrieve approved documents or use embeddings in the inspected flow. RAG, citations and pgvector are planning proposals, not implemented capabilities.

Agent 2: deterministic eligibility filtering

How used: Compares confirmed full-care/stabilization needs with hospital capabilities, receiving status and selected resource/staff checks.

Why it fits: Eligibility should be explainable and reproducible. A hospital should not be declared capable because a language model produced a persuasive sentence.

Technique: Constraint filtering before ranking. Missing capability/resource reasons are returned for inspection.

Limit: The evaluator covers explicit mappings implemented in code; it is not a comprehensive clinical rules engine. Acceptance still needs to recheck resources because state can change after recommendation.

Agent 3: time-to-care and stabilization priorities

The current source includes `FULL_CARE_AVAILABLE`, `STABILIZATION_ONLY` and `NO_ELIGIBLE_DESTINATION` modes. Full-care candidates are sorted by time-to-care; stabilization-only candidates use road travel time. The UI can therefore avoid an empty A card blocking an available stabilization option.

The conceptual calculation is:

```
care start = max(arrival time, resource ready time) + receiving delay
time to care = care start - evaluation time
```

In the inspected graph-ranking implementation, resource readiness is currently set to zero. Do not describe it as reading a real operating-theatre schedule. Normal Agent 3 ranking still calls the graph-node routing interface; separate geographic-route code existing elsewhere does not prove every recommendation uses Google traffic.

Why it fits: A short drive is not the only relevant delay. Keeping travel and receiving delay separate makes ranking explainable. Stabilization priority is a distinct objective, not a claim that a backup has full definitive-care capabilities.

Deterioration preview and human confirmation

The source now contains a deterioration dialog, a diversion-preview endpoint and explicit care-objective fields. They support comparing alternatives before a user changes the destination.

Why it fits: A preview can expose route/eligibility information without immediately performing a consequential mutation. The crew retains the decision boundary.

Limit: Presence of these paths is not proof that every freshness, idempotency or race requirement from the earlier plan is complete. Explain the current workflow without claiming fully verified emergency automation.

7. Maps, routing and simulation

Google Maps JavaScript API

`LiveMap.tsx` now loads the Google Maps script when a browser key exists and creates map, marker, polyline and traffic-layer objects. It retains an SVG fallback for unavailable SDK/test conditions.

Why used: A real basemap provides recognizable streets and geographic context. Route polylines and hospital markers show where the coordinated journey is occurring.

Limit: A basemap is not a GPS source, and a traffic layer is not the same as recalculating a hospital recommendation. Keys and live connectivity were not verified in this document task. The current marker code uses `google.maps.Marker`; do not describe it as an Advanced Marker implementation.

Google Routes API and polyline decoding

The backend adapter sends Compute Routes requests and decodes encoded route geometry into coordinate points. Distance, duration and geometry can then be delivered to the frontend. These response fields are described in Google's Compute Routes guide.

Why used: Road-following route estimates are more meaningful than a straight line between latitude/longitude points. Backend routing also keeps server credentials out of browser application code.

Limit: Provider failure/fallback provenance needs to stay visible. A provider enum such as `LIVE_OSRM` does not by itself demonstrate a real OSRM integration. Distinguish map, route and position sources.

Dijkstra shortest-path search

The simulated router constructs a graph from nodes and edges and searches for the minimum weighted travel duration. It retains predecessor nodes/edge IDs to reconstruct the selected route.

Why used: Dijkstra is deterministic for nonnegative edge weights and well suited to a small demo road graph. Traffic multipliers can increase edge costs without introducing a model dependency.

Current implementation detail: The router uses a scan over unvisited nodes rather than a priority queue, so the node-selection work is quadratic in the number of nodes. That is acceptable for the small fixture graph, but it should not be presented as a large-city routing engine.

Traffic weighting and route provenance

The simulation adjusts travel costs/speeds with edge multipliers. Route sources distinguish graph estimates and external-provider results. This enables demonstrations of congestion and alternative destinations without requiring paid APIs for every test.

The limitation is geographic scope: a small seeded graph does not establish a valid route for arbitrary real GPS positions. Current fixture/reset coordinates still use the earlier region; NITK relocation remains planned.

Server-side simulation

`simulation/service.ts` holds run state, a ticker, active route and segment progress. It supports play, pause, speed changes, stepping, traffic/hospital scenarios and telemetry broadcast. Current movement consumes time across multiple directed segments rather than treating each update as a whole-route jump.

Why used: The server supplies a common state to all viewers. Demo behavior does not depend on an individual browser's animation speed.

Limits: The timer measures elapsed time with `Date.now()` despite a monotonic-clock comment; it is not a true monotonic source. A checkpoint counter is not a persistent replay log. The service remains process-local and should not be described as a distributed simulation scheduler.

SVG, isometric view and declared 3D libraries

SVG renders scalable roads, nodes, labels and markers. The current simulator's apparent 3D effect comes from CSS perspective applied to SVG; it is an isometric-style visualization, not a rendered Three.js city.

Three.js, Fiber and Drei are declared for potential 3D work. Their intended roles are geometry/rendering, React scene integration and reusable controls/helpers. The declared React 18/Fiber 9 pairing needs attention: Fiber's compatibility guidance pairs Fiber 8 with React 18 and Fiber 9 with React 19.

Buildings, trees, projected NITK geometry and a genuine 3D scene belong to the future implementation plan, not the current demo description.

8. Data, consistency and realtime coordination

JavaScript Maps as the current business store

`db/store.ts` stores users, hospitals, cases, assessments, destinations, resource pools, allocations and recommendations in Maps. Audit/outbox entries are arrays.

Why this fits a prototype: Fast setup, deterministic seeded demos and easy isolated tests without requiring a working database for every interaction.

Tradeoff: Memory is not durable storage. Process restart loses runtime changes unless explicitly reconstructed. Separate replicas would have separate state. The word “database” in a class comment does not create database transactions.

PostgreSQL 16, pg and SQL migrations

The Compose configuration provides PostgreSQL 16. The `pg` library provides a connection pool, and the repository contains an initial SQL schema and migration tooling. The schema uses primary/foreign keys, unique constraints, checks, timestamps and JSONB for structured observations.

Why this technology fits the target domain: Related entities and resource reservations need exact records and transactional constraints. SQL can protect relationships and counts more directly than document similarity search.

Current boundary: The inspected primary business store does not implement its normal operations through `pg` transactions. Database connection health and a schema file should be presented as infrastructure present, not proof that every case is persisted.

Resource accounting and all-or-nothing checks

```
available = total - out of service - external occupancy
            - reserved - in use
```

Acceptance checks required resource/staff pools before applying reservations in the current process. Standby requests do not reserve scarce resources. Explicit care objectives choose full-care or stabilization needs. Arrival, handover and equipment release are separate operations.

Why used: Capacity should not become negative or remain locked forever after a diversion. Separating reservation from actual use explains why arriving and releasing equipment are not the same event.

Limit: Synchronous precheck-and-apply behavior in one process is not equivalent to a durable multi-process SQL transaction. Do not claim production atomicity solely from comments containing “atomic commit”.

Optimistic version checks

Resource, assessment and destination versions help reject stale updates. A request can include the version the user saw; the server rejects incompatible newer state instead of silently overwriting it.

Why used: A hospital operator and ambulance crew may act on the same case at different times. Version checks surface conflicts rather than assuming the first screen is still current.

Limit: Version checks need to be applied at every relevant write boundary. Their presence in some operations is not a universal concurrency guarantee.

Socket.IO - declared 4.8.4

Server and client exchange named events for cases, destinations, inventory and telemetry. The client uses websocket/polling transports and reconnect options. Socket.IO is an event-based bidirectional communication library, not just a synonym for raw WebSocket. See its official introduction.

Why used: A receiving hospital should see an incoming request without manual refresh, and the ambulance should see updated destination state promptly.

Limit: Reconnection does not automatically reconstruct every missed business event. State snapshots, ordering and authorization still matter. The inspected realtime code includes permissive demo connection/broadcast behavior; do not claim complete tenant isolation.

Audit and outbox patterns

Audit entries record actor/action/entity details. Outbox-like entries separate recording a domain event from notifying interested consumers; some routes also emit Socket.IO events directly.

Why used: The project needs to explain who changed a destination or resource. Event records also create a place to support reliable publication later.

Current boundary: These are in-memory collections. They are not a durable transactional outbox or a tamper-evident audit ledger. Retry delivery and deduplication should not be assumed without verification.

9. Testing, build and deployment tools

Vitest, Testing Library, jest-dom and jsdom

Vitest is the declared test runner. Testing Library exercises UI through rendered elements and interactions; jest-dom provides DOM assertions; jsdom supplies a browser-like test environment. Fastify's `app.inject` lets API tests call the app without starting a separate public server.

Why used: The project has both stateful backend transitions and interactive frontend workflows. Testing these boundaries is more valuable than testing only rendering snapshots.

The repository includes suites for authentication, routing, hospital resources, dispatch, diversion, simulation, maps and stabilization/deterioration. This guide does not report a passing count because the suites were not run for this task. Mocked Google/browser objects do not prove that a live map key or real GPS device works.

pnpm workspaces and TypeScript builds

pnpm runs scripts across the apps and shared packages. Workspace dependencies point to local contracts and fixtures. The root exposes development, build, test and typecheck commands.

Why used: Shared code can evolve alongside its consumers without publishing a package for every change. Separate test and typecheck commands distinguish behavioral checks from compile-time checks.

Limit: Current lint scripts are placeholders that print a message. Do not present them as an ESLint/static-quality pipeline.

Docker, Compose and Nginx

Dockerfiles describe separate API and web builds. Compose describes database, API and web services with networking and health/dependency configuration. Nginx serves frontend assets, forwards `/api/` to the API and supplies WebSocket upgrade headers for `/socket.io/`.

Why used: Containers give the team a repeatable service layout. Nginx presents frontend assets and backend paths through one entry point and supports direct navigation through the SPA fallback.

Limit: Deployment files existing in source do not prove the containers currently build or production security is complete. Development credential defaults, CORS settings, TLS termination, secrets and migration execution need deployment review. No deployment was performed for this document.

Health checks and structured errors

Health endpoints, database ping checks and an API status badge help distinguish a running web page from a healthy backend dependency. `ApiError` carries status/code/message information back to the UI.

Why used: A visible “database degraded” state is more informative than treating every failure as a generic network error. Health checks are diagnostics, not proof of correct business data or fully persistent storage.

10. Current versus planned: presentation checklist

Capability	Current evidence	How to describe it
React/Fastify typed application	Active frontend/API modules	Implemented prototype architecture
Gemini interpretation	Direct provider adapter plus rules fallback	AI-assisted proposals; live provider use depends on configuration
Stabilization priority and deterioration	Contracts, UI dialog and backend paths present	Current workflow implementation, with validation/testing still requiring evidence
Google basemap and routes	Browser SDK integration and backend Routes adapter	Integrated in source; not verified live during this review
Graph routing	Dijkstra over fixtures	Deterministic demo routing
Device GPS	Browser geolocation and position endpoint	Opt-in device tracking path, not guaranteed background fleet telemetry
Simulator	Server movement plus SVG view	Server-driven simulation with isometric-style display
PostgreSQL persistence	Schema, pool, migrations, Compose	Database infrastructure; main business store remains in memory
True 3D city	Libraries declared, SVG renderer remains	Planned renderer upgrade
NITK regional dataset	Earlier map/reset coordinates remain	Planned reviewed geography migration
Elective procedure deferral tickets	Planning document; no inspected runtime scheduling module	Planned hospital-reviewed coordination feature
RAG and vector database	Planning document; no retrieval/embedding runtime found	Proposed future Agent 1 upgrade

Why the combination makes sense

React and Tailwind address the interface. TypeScript and Zod reduce contract mistakes. Fastify organizes API boundaries. Gemini assists with language interpretation, while deterministic code handles eligibility, ranking and resource state. Google services supply geographic context and external routes; the graph simulator provides a controlled demonstration path. Socket.IO shares changes across roles. PostgreSQL is appropriate for the durable transactional layer, but that integration must be represented honestly as incomplete in the current business store.

This is a reasoned fit for the project, not proof that each technology is universally better than every alternative. The project's value comes from how the boundaries work together, not the number of libraries listed.

11. Explain the project in an interview or demonstration

One-minute explanation

“GoldenHour is a full-stack TypeScript prototype for coordinating an ambulance with receiving hospitals. React provides the role-specific dashboards, Fastify exposes the APIs, and shared Zod schemas keep request shapes consistent. Agent 1 uses Gemini with a rules fallback to propose requirements for human review. Agent 2 checks hospital eligibility, and Agent 3 ranks destinations using travel and receiving delay. The app supports stabilization options when full care is unavailable and a crew-reviewed deterioration diversion flow. Socket.IO distributes updates, while Google Maps and Routes provide geographic integration alongside a deterministic graph simulator. The current business store is mainly in memory; durable PostgreSQL operations, RAG and the richer NITK 3D/scheduling features are further work.”

Common questions and accurate answers

Are all three agents LLMs? No. Agent 1 has an LLM-backed interpreter. Eligibility and ranking are explicit algorithms, which makes their decisions easier to test and inspect.

Why not let Gemini select and reserve the hospital? Generated language should not mutate scarce resources. Confirmed requirements feed deterministic checks, and authorized workflow actions control requests and allocations.

Why both TypeScript and Zod? TypeScript checks source code before execution. Zod checks external values at runtime. They address different failure points.

Why not use only a vector database? Similarity search is for finding related knowledge. It cannot replace exact inventory counts or reservation transactions. No vector database is currently used by Agent 1.

Is the simulator really 3D? Its current rendered view is SVG with perspective styling. Three.js-related dependencies are declared, but a true 3D environment is still a proposed upgrade.

Does the map key provide the ambulance location? No. The map renders geography; device GPS or simulation supplies position; the route service estimates a path and travel time.

Is the database already fully durable? PostgreSQL support is present, but the main inspected store uses Maps. Do not claim persistence across restart or multi-server transactional correctness without implementing and testing it.

What does the current testing prove? Source contains meaningful test suites. A presentation should quote results from an actual recent run, not infer success from filenames or old documentation.

12. Source map and supporting references

Repository evidence

- `package.json`, `pnpm-workspace.yaml` and package manifests: declared tools and scripts.
- `apps/web/src/App.tsx`, pages, `context/AuthContext.tsx`, `lib/api-client.ts`: UI routing, auth state and HTTP behavior.
- `apps/web/src/components/LiveMap.tsx` and `Simulation3DView.tsx`: geographic SDK versus current SVG simulator.
- `apps/api/src/server.ts`, `config.ts`, `modules/auth/`: server assembly, configuration and session/password handling.
- `apps/api/src/modules/agents/`: requirement interpretation, filtering and ranking.
- `apps/api/src/modules/routing/` and `modules/simulation/`: provider adapters, graph search and server movement.
- `apps/api/src/modules/dispatch/routes.ts`, `db/store.ts`, `modules/realtime/index.ts`: preview/actions, resource state and event behavior.
- `packages/contracts/src/`, `packages/scenario-fixtures/src/`: shared shapes and demo inputs.
- `migrations/001_initial_schema.sql`, `db/index.ts`, `db/migrate.ts`: PostgreSQL infrastructure.
- `apps/api/test/`, `apps/web/src/test/`: test coverage areas, without a fresh execution claim.
- Dockerfiles, Compose files, `nginx.conf`, `apps/web/vite.config.ts`: deployment and development wiring.

Primary documentation

- Socket.IO introduction: event-based communication and transports.
- Node.js crypto: password derivation, randomness and comparison primitives.
- Google Compute Routes: route duration, distance and geometry.
- React Three Fiber introduction: renderer role and React compatibility.

Review boundary: This is a documentation artifact based on the current inspected source. It is not a benchmark, deployment certificate, security audit or clinical validation. No application code was changed to create it.